最优化方法上机作业实验报告

解晟平 2300010702

2025 年 12 月 22 日

1 问题描述

考虑如下 Group LASSO 问题:

$$\min_{x\in\mathbb{R}^{n\times l}} f_\mu(x) = \frac{1}{2}\|Ax-b\|_F^2 + \mu\|x\|_{1,2} \tag{1.1}$$

这里 $A\in\mathbb{R}^{m\times n}$, $b\in\mathbb{R}^{m\times l}$, 为给定的系数，$\mu$ 为正则化系数。$\|x\|_{1,2}$ 表示矩阵的 $L_{1,2}$ 范数，也即

$$\|x\|_{1,2} = \sum_{i=1}^{n}\|x(i,1:l)\|_2$$

其中 $x(i,1:l)$ 表示矩阵 x 的第 i 行。

2 算法实现

2.1 使用 CVXPY 求解

我们将使用 Python 作为本次作业的编程语言。针对最优化问题的求解，Python 提供了 CVXPY 软件包，它支持将输入的优化问题自动转换为后端求解器可以接受的形式，从多种常用的优化问题求解器中自动匹配或手动置顶与问题类型相对应的求解器，并调用求解器接口完成求解。

在本次作业中, 我们使用 CVXPY 调用 Mosek 和 Gurobi 求解器求解 Group LASSO 问题。Mosek 和 Gurobi 是作为被广泛使用的商业优化求解器，内置多种启发式算法和加速策略，相比于开源的求解器，在处理凸优化问题时往往能够更加快速的收敛到高精度解。因此我们将这两个求解器求解的结果作为后续对比各个优化算法的基准线。

这部分的具体实现位于 `cvx_mosek.py` 以及`cvx_gurobi.py`

2.2 直接调用求解器求解

2.2.1 使用 Mosek 求解

我们不使用 CVXPY, 而是直接调用 Mosek 求解器。由于 Mosek 求解器只适用于求解标准的二次锥规划问题, 因此我们需要将问题(1.1)转换为如下的 SOCP 形式:

$$
\begin{aligned}
\min_{s,t} \quad & \frac{1}{2}t + \mu(s_1 + s_2 + \cdots + s_n) \\
\text{s.t.} \quad & \begin{cases} \|Ax - b\|_F^2 \leq 2 \cdot t \cdot t_0 \\ t_0 = \frac{1}{2} \\ \|x(i,:)\|_2 \leq s_i,\ i = 1, 2, \cdots, n \end{cases}
\end{aligned}
$$

这部分的具体实现位于 `exact_mosek.py`

2.2.2 Gurobi

由于 Gurobi 适用于添加二次约束, 因此在使用 Gurobi 求解器进行求解, 我们把 Group LASSO 问题转换为如下 QCP 形式:

$$
\begin{aligned}
\min_{x,z} \quad & \frac{1}{2}t^2 + \mu(s_1 + s_2 + ... + s_n) \\
\text{s.t.} \quad & \begin{cases} \|Ax - b\|_F^2 \leq t^2 \\ \|x(i,:)\|_2^2 \leq s_i^2,\ i = 1, 2, \cdots, n \\ t \geq 0, s_i \geq 0,\ i = 1, 2, \cdots, n \end{cases}
\end{aligned}
$$

这部分的具体实现位于 `gurobi.py`

2.3 输出信息

对于基于梯度的方法，我们在优化过程中，通过 out 字典来输出如下信息：

- Fval：算法结束时的目标函数值
- obj_val: 记录算法迭代过程中目标函数值的变化的列表
- grad_norm: 记录算法迭代过程中梯度的 Frobenius 范数的变化的列表

对于基于对偶问题的方法，我们在优化过程中，通过 out 字典来输出如下信息：

- Fval：算法结束时的目标函数值
- obj_val: 记录算法迭代过程中目标函数值的变化的列表
- dual_gap: 记录算法迭代过程中对偶间隙的变化的列表

对于其他方法，我们在优化过程中，通过 out 字典来输出如下信息：

- Fval：算法结束时的目标函数值
- obj_val: 记录算法迭代过程中目标函数值的变化的列表

2.4 次梯度法和光滑化的梯度法

2.4.1 次梯度法

由于 L_2 范数 $\|x\|_2$ 在 $x=0$ 处不可微，我们首先选用次梯度法进行求解。计算函数的次梯度为：

$$\partial f(x) = A^T(Ax-b) + \mu\partial\|x\|_{1,2}$$

这里 $\partial\|x\|_{1,2}$ 为 $L_{1,2}$ 范数的次梯度，具体的，我们取如下次梯度：

$$\partial\|x\|_{1,2}(i,:) = \begin{cases} \frac{x(i,:)}{\|x(i,:)\|_2}, & \|x(i,:)\|_2 \neq 0, \\ 0, & \|x(i,:)\|_2 = 0, \end{cases}$$

这里 $(i,:)$ 表示第 i 行。由于 Group-Lasso 包含的线性规划问题本身往往是过参数化的，也即 $m<n$, 而优化过程更着重于寻找 Lasso 问题的稀疏解。因此，在训练初期，为了加速模型向稀疏的参数空间收敛的速度，我们可以将原先的正则化系数 μ 替换为更大的值 $\mu' = c\mu$, 对新的目标函数 $f_{\mu'}(x)$ 进行一定步数的迭代，随后按比例将 c 的值减小到 1，最后再求解原问题。具体的算法如下：

Algorithm 1 改进的次梯度算法

超参数: 缩放系数 $\gamma > 1$, 预循环数量 s, 单循环最大步数 k_{max}, 总最大迭代步数 t_{max}, 初始学习率 η_0, 学习率调度器 l.

```
t ← 0
for m = s, s − 1, ··· , 1 do
    μ′ ← γ^m μ
    k ← 0
    while not converge and k < k_max do
        η_t = l(η_0, k)                    ▷ 根据学习率调度器计算步长
        g_t = ∂f_{μ′}(x)                   ▷ 对扩大正则项的函数求次梯度
        x ← x − η_t · g_t                  ▷ 更新 x
        k ← k + 1
        t ← t + 1
    end while
end for        ▷ 扩大正则化项的优化阶段结束，对原函数继续优化 k ← 0
while not converge and t < t_max do
    η_t = l(η_0, k)                        ▷ 根据学习率调度器计算步长
    g_t = ∂f_μ(x)                          ▷ 对原函数求次梯度
    x ← x − η_t · g_t                      ▷ 更新 x
    k ← k + 1
    t ← t + 1
end while
return x
```

2.4.2 光滑化的梯度法

为了使用梯度法求解问题, 我们可以对目标函数进行光滑化近似, 使其处处可微. 为此, 将目标函数中的 L_2 范数替换为其光滑化近似函数:

$$l_\sigma(x) = \begin{cases} \|x\|_2^2/2\sigma, & |x| \le \sigma, \\ \|x\|_2 - \sigma/2, & |x| \ge \sigma, \end{cases}$$

其中 $\sigma > 0$ 是一个参数, 并随着迭代步数的增加逐渐减小. 对于 $l_\sigma(x)$, 我们可以计算其梯度

$$\nabla l_\sigma(x) = \begin{cases} x/\sigma, & |x| \le \sigma, \\ x/\|x\|_2, & |x| \ge \sigma, \end{cases}$$

更改后，目标函数变为

$$f_{\mu,\sigma}(x) = \frac{1}{2}\|Ax - b\|_F^2 + \mu \sum_{i=1}^{n} L_\sigma(x(i,:))$$

我们采用与次梯度法类似的连续化策略，并将 σ 同 μ 等比例放大，以确保在 0 附近目标函数有较好的平滑性。具体的算法如下：

2.4.3 步长选取

通过对梯度方法收敛性的分析结果可知, 初始学习率 η_0 应接近 $1/L$, 这里 L 为目标函数的 Lipschitz 常数，这里我们近似为 Lasso 项的 Lipschitz 常数 $L_0 = \|A\|_2^2 = \lambda_{max}(A^T A)$; 学习率应随时间 T 进行 $\sqrt{1/T}$ 的衰减，但为了加快收敛速度，我们在每个循环开始时先使用恒定步长，随后再进行衰减，最终得到的学习率调度器如下：

$$l(\eta_0, k) = \begin{cases} \eta_0, & k \le k_0, \\ \frac{\eta_0}{\sqrt{k-k_0}}, & k > k_0, \end{cases}$$

并令 $\eta_0 = \frac{1}{\|A\|_2^2}$。

上述算法的代码实现见`SubGradient.py` 以及`GD.py`

Algorithm 2 改进的梯度下降算法

超参数: 缩放系数 $\gamma > 1$, 预循环数量 s, 单循环最大步数 k_{max}, 总最大迭代步数 t_{max}, 初始学习率 η_0, 平滑化参数 σ, 学习率调度器 l.

```
t ← 0
for m = s, s − 1, ··· , 1 do
    μ′ ← γ^m μ
    σ′ ← γ^m σ
    k ← 0
    while not converge and k < k_max do
        η_t = l(η_0, k)                    ▷ 根据学习率调度器计算步长
        g_t = ∇f_{μ′,σ′}(x)                ▷ 对扩大正则项并平滑化的函数求梯度
        x ← x − η_t · g_t                  ▷ 更新 x
        k ← k + 1
        t ← t + 1
    end while
end for                ▷ 扩大正则化项的优化阶段结束，对原函数继续优化
k ← 0
while not converge and t < t_max do
    η_t = l(η_0, k)                        ▷ 根据学习率调度器计算步长
    g_t = ∇f_{μ,σ}(x)                      ▷ 对平滑化的原函数求梯度
    x ← x − η_t · g_t                      ▷ 更新 x
    k ← k + 1
    t ← t + 1
end while
return x
```

2.5 近似点梯度法和 Nesterov 加速

2.5.1 近似点梯度法

由于问题(1.1)由一个可微凸函数 $g(x) = \|Ax - b\|_F^2$ 和一个不可微闭凸函数 $h(x) = \|x\|_{1,2}$ 组成, 因此我们可以使用近似点梯度法进行求解. 对函数 $f_\mu(x)$ 使用近似点梯度法，单步迭代的具体形式如下:

$$x_{t+1} = \mathrm{prox}_{\eta_t \mu \cdot h}(x_t - \eta_t \cdot \nabla g(x_t))$$

其中用邻近算子 $\mathrm{prox}_h(x)$ 的定义如下:

$$\mathrm{prox}_h(x) = \arg\min_y \left\{ \frac{1}{2}\|y - x\|_2^2 + h(y) \right\}$$

针对 $h(x) = \|x\|_{1,2}$, 我们可以计算 $\mathrm{prox}_{th}(x)$ 的闭式解:

$$\mathrm{prox}_{kh}(x)(i,:) = \begin{cases} \left(1 - \frac{k}{\|x(i,:)\|_2}\right) x, & \|x(i,:)\|_2 \geq k, \\ 0, & \|x(i,:)\|_2 < k, \end{cases} \tag{2.1}$$

我们一样选择类似的学习率调度器与连续化策略进行加速 (即在开始时用更大的 μ' 替代 μ，因此近似点梯度法的框架与算法 1 类似, 只是在更新 x 时使用了邻近算子代替了次梯度.

2.5.2 Nesterov 加速

在近似点梯度法的基础上，我们可以使用 Nesterov 动量算法进行加速. 具体迭代公式如下:

$$y = x_t + \frac{t-1}{t+2}(x_t - x_{t-1})$$
$$x_{t+1} = \mathrm{prox}_{\eta_t \mu \cdot h}(y - \eta_t \nabla g(y))$$

在上式中我们约定 $x_{-1} = x_0$. 这里我们仍然使用连续化策略，但由于不同循环的目标函数不同，因此在每个循环内将对动量进行重置。具体算法见算法3:

上述算法的代码实现见`ProximalGD.py` 以及`FastProximalGD.py`

Algorithm 3 改进的加速近似点梯度算法

超参数: 缩放系数 $\gamma > 1$, 预循环数量 s, 单循环最大步数 k_{max}, 总最大迭代步数 t_{max}, 初始学习率 η_0, 学习率调度器 l.

$t \leftarrow 0$
for $m = s, s-1, \cdots, 1$ **do**
  $\mu' \leftarrow \gamma^m \mu$
  $k \leftarrow 0$
  while not converge and $k < k_{\max}$ **do**
    $\eta_t = l(\eta_0, k)$ ▷ 根据学习率调度器计算步长
    $y \leftarrow x_t + \frac{k-1}{k+2}(x_t - x_{t-1})$ ▷ 计算 Nesterov 动量
    $x_{t+1} \leftarrow \mathrm{prox}_{\eta_t \mu' \cdot h}(y - \eta_t \nabla g(y))$ ▷ 对扩大正则项的函数使用近似点梯度法 x
    $k \leftarrow k + 1$
    $t \leftarrow t + 1$
  end while
end for ▷ 扩大正则化项的优化阶段结束，对原函数继续优化
$k \leftarrow 0$
while not converge and $t < t_{\max}$ **do**
  $\eta_t = l(\eta_0, k)$ ▷ 根据学习率调度器计算步长
  $y \leftarrow x_t + \frac{k-1}{k+2}(x_t - x_{t-1})$ ▷ 计算 Nesterov 动量
  $x_{t+1} \leftarrow \mathrm{prox}_{\eta_t \mu' \cdot h}(y - \eta_t \nabla g(y))$ ▷ 对原函数使用近似点梯度法 x
  $k \leftarrow k + 1$
  $t \leftarrow t + 1$
end while
return x_t

2.6 增广拉格朗日函数法

在这一部分中, 我们对问题(1.1)使用增广拉格朗日函数法 (ALM) 进行求解. 问题(1.1)的对偶问题为:

$$\begin{aligned} \min_{Y\in\mathbb{R}^{m\times l}} \quad & \langle b, Y\rangle + \frac{1}{2}\|Y\|_F^2 \\ \text{s.t.} \quad & \|A^TY\|_{\infty,2} \le \mu \end{aligned} \tag{2.2}$$

为了使用增广拉格朗日函数法求解这个问题, 我们把对偶问题改写为如下的形式:

$$\begin{aligned} \min_{Y,Z} \quad & \langle b, Y\rangle + \frac{1}{2}\|Y\|_F^2 \\ \text{s.t.} \quad & A^TY = Z \\ & \|Z\|_{\infty,2} \le \mu \end{aligned} \tag{2.3}$$

然后, 构造如下的增广拉格朗日函数:

$$L_\sigma(Y,Z;X) = \langle b, Y\rangle + \frac{1}{2}\|Y\|_F^2 + \left\langle X, A^TY - Z\right\rangle + \frac{\sigma}{2}\|A^TY - Z\|_F^2, \quad \|Z\|_{\infty,2} \le \mu \tag{2.4}$$

对于增广拉格朗日函数 $L_\sigma(Y,Z;X)$, 我们要求在约束条件下 $\|Z\|_{\infty,2} \le \mu$ 下的最小值，由于将 $L_\sigma(Y,Z;X)$ 看成关于 Y,Z 各自的函数时，在全参数空间下均可以求得闭式解，具体解的形式如下：

$$\begin{aligned} Y' &= \arg\min_Y L_\sigma(Y,Z;X) = \left(\sigma AA^T + I_m\right)^{-1}\left[A\left(\sigma Z - X\right) - b\right] \\ Z_0' &= \arg\min_Z L_\sigma(Y,Z;X) = \frac{X}{\sigma} + A^TY \end{aligned}$$

因此我们可以使用坐标下降法，特别的，对于 Z 的更新，我们需要再将闭式解投影到约束球 $\|Z\|_{\infty,2} \le \mu$ 上，具体如下:

$$Z' = \arg\min_{\|Z\|_{\infty,2}\le\mu} \|Z - Z_0\|_F^2$$

$$\Longrightarrow \; Z'(i,:) = \begin{cases} Z_0'(i,:), & \|Z_0'(i,:)\|_2 \le \mu, \\ \mu\frac{Z_0'(i,:)}{\|Z_0'(i,:)\|_2}, & \|Z_0'(i,:)\|_2 \ge \mu, \end{cases}$$

因此，总结上述迭代过程，我们可以得到对应的增广拉格朗日函数法, 见算法(2.6):

Algorithm 4 使用分块坐标下降的增广拉格朗日函数法

超参数: 罚因子 σ, 子问题最大求解步数 k_{max}, 外循环最大迭代步数 t_{max}, 内循环最大迭代步数 k_{max}, 外循环终止条件 tol, 内循环终止条件 tol_i, 更新步长 η.

```
Y ← Y_0, Z ← Z_0, X ← X_0, t ← 0                    ▷ 初始化变量和乘子
while t < t_max and ‖A^T Y − Z‖_F > tol do
    k ← 0
    while k < k_max and ‖Z − Z_old‖_F > tol_i do
        Z_old ← Z
        Y ← arg min_Y L_σ(Y, Z; X)                  ▷ 更新 Y
        Z̃ ← arg min_Z L_σ(Y, Z; X)                  ▷ 更新 Z
        Z ← arg min_{‖Z‖_{∞,2} ≤ μ} ‖Z − Z̃‖_F^2     ▷ 做投影使得满足约束
        k ← k + 1
    end while
    t ← t + 1
    X ← X + η · σ(A^T Y − Z)                        ▷ 更新乘子
end while
return −X   ▷ 在构造的增广拉格朗日函数中, X 与原问题的解差一个符号
```

上述算法的代码实现见`ALM_dual.py`

2.7 交替方向乘子法

2.7.1 原问题的交替方向乘子法

原问题(1.1)是一个分成两块的无约束优化问题，可以转化为如下适用于交替方向乘子法（ADMM) 的形式:

$$\begin{aligned} \min_{x,y\in\mathbb{R}^{n\times l}} \quad & \frac{1}{2}\|Ax-b\|_F^2+\mu\|y\|_{1,2} \\ \text{s.t.} \quad & x=y \end{aligned}$$

构造增广拉格朗日函数为:

$$L_\rho(x,y;\lambda)=\frac{1}{2}\|Ax-b\|_F^2+\mu\|y\|_{1,2}+\langle\lambda,x-y\rangle+\frac{\rho}{2}\|x-y\|_F^2$$

计算交替方向乘子的更新时，对变量 x 可以直接求得闭式解:

$$x'=\arg\min_x L_\rho(x,y;\lambda)=\left(A^TA+\rho I_n\right)^{-1}\left(A^Tb+\rho y-\lambda\right)$$

对 y 求极小值时, 相当于做一步近似点梯度下降，具体如下:

$$\begin{aligned} z &= x+\frac{1}{\rho}\lambda \\ y' &= \arg\min_y L_\rho(x,y;\lambda)=\arg\min_y \mu\|y\|_{1,2}+\frac{\rho}{2}\|y-z\|_F^2 \end{aligned}$$

因此 $y'=\mathrm{prox}_{\frac{\rho}{\mu}h}(z)$, 具体的表达式同(2.1)。综上所述，我们得到 ADMM 算法如下:

2.7.2 对偶问题的交替方向乘子法

原问题的对偶问题(2.3)可以改写为如下适用 ADMM 的形式:

$$\begin{aligned} \min_{Y,Z} \quad & \langle b,Y\rangle+\frac{1}{2}\|Y\|_F^2+\mathcal{I}_{\mathcal{A}}(Z) \\ \text{s.t.} \quad & A^TY=Z \end{aligned} \tag{2.3}$$

这里 $\mathcal{I}_{\mathcal{A}}(\cdot)$ 表示集合 $\mathcal{A}=\left\{Z\middle|\|Z\|_{\infty,2}\le\mu\right\}$ 的示性函数。对该函数使用交替方向乘子法等价于对问题(2.4)中的 Y,Z 各做一次坐标下降，也即此时的

Algorithm 5 对原函数的交替方向乘子法

超参数: 罚因子 ρ, 终止条件 tol, 更新步长 η.

$x \leftarrow x_0$, $y \leftarrow y_0$, $\lambda \leftarrow \lambda_0$

while $\|x - y\|_F \geq tol$ **do**

  $x \leftarrow \arg\min_x L_\rho(x, y; \lambda)$ ▷ 更新 x

  $y \leftarrow \arg\min_y L_\rho(x, y; \lambda)$ ▷ 更新 y

  $\lambda \leftarrow \lambda + \eta \cdot \rho(x - y)$ ▷ 更新乘子

end while

return x

算法即为在算法2.6中取 $k_{max} = 1$ 的情形。

上述算法的代码实现见`ADMM_primal.py`以及`ADMM_dual.py`

3 实验结果

3.1 评价指标

我们使用多种指标来评价优化算法的性能:

- 目标函数值 (Fval): 算法停止时的目标函数值。
- 算法求得的解与 Mosek 求得的解的误差 (ErrMosek): 使用公式 $\frac{\|x - x_{\text{mosek}}\|_F}{1 + \|x_{\text{mosek}}\|_F}$ 计算.
- 算法求得的解与 Gurobi 求得的解的误差 (ErrGurobi): 使用公式 $\frac{\|x - x_{\text{gurobi}}\|_F}{1 + \|x_{\text{gurobi}}\|_F}$ 计算.
- 算法求得的的解与问题最优解的误差 (ErrExact): 使用公式 $\frac{\|x - x_{\text{opt}}\|_F}{1 + \|x_{\text{opt}}\|_F}$ 计算.
- 解的稀疏度 (Sparsity): 在这个问题中, 解的稀疏度定义为解中绝对值大于解的 l_∞ 范数的 $1e - 6$ 倍的参数占总参数量的比例. 数值越大说明解越稀疏。
- 算法运行时间 (Time): 为了精准估计计算算法单次运行的时间，我们需要排除运行系统级指令和启动算法，分配内存等的“冷启动”时间，

因此，我们首先运行算法 20 次作为预热，随后再运行算法 50 次，用这 50 次的平均时间作为对运行时间的估计。

- 算法迭代次数 (Iter): 算法迭代的次数, 用于评价算法的收敛速度.

3.2 运行结果及分析

各算法的测试结果如表 1 所示.

Algorithm	Fval	ErrMosek	ErrGurobi	ErrExact	Sparsity	Time(ms)	Iter
CVX-Mosek	6.70575e-01	0.00e+00	5.36e-07	3.19e-05	0.103	304.0677	9
CVX-Gurobi	6.70575e-01	5.36e-07	0.00e+00	3.23e-05	0.104	752.2501	10
Mosek	6.70575e-01	1.79e-07	6.46e-07	3.18e-05	0.103	189.2986	11
Gurobi	0.670588e-01	3.16e-05	3.21e-05	3.55e-07	0.100	1923.8127	7
Subgradient	6.70575e-01	2.96e-06	2.90e-06	3.33e-05	0.100	541.1530	1887
Smoothed-GD	6.70575e-01	2.96e-06	2.90e-06	3.33e-05	0.100	230.6406	606
ProxGD	6.70575e-01	2.96e-06	2.90e-06	3.33e-05	0.100	200.7275	662
Fast-ProxGD	6.70575e-01	1.97e-06	2.12e-06	3.22e-05	0.100	80.3467	207
ALM-Dual	6.70598e-01	4.20e-05	4.17e-05	6.71e-05	0.100	59.2430	39
ADMM-Primal	6.70575e-01	2.29e-07	7.14e-07	3.18e-05	0.103	289.3226	1375
ADMM-Dual	6.70588e-01	3.06e-05	3.02e-05	5.88e-05	0.100	27.7792	67

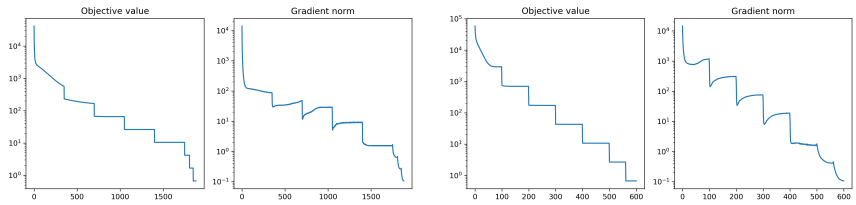
表 1: 不同算法的测试结果

接下来我们将展示优化过程的详细曲线。首先我们对每种算法均展示目标函数值的变化曲线，对于梯度方法，我们一并展示梯度的 F 范数作为参考指标，对于基于对偶问题的算法，我们一并展示对偶间隙作为参考指标。

3.2.1 次梯度法和光滑化的梯度法

从结果中可以看出, 次梯度法和光滑化的梯度法无论从收敛速度还是解的系数行上均不如其他算法。这可能是由于对于 0 点处不可微的函数使用梯度法与函数的损失地形图不匹配，而其与优化算法通过计算子优化问题的闭式解，更加适配与问题(1.1)。通过这两种方法内部的对比，我们发现使用光滑的函数近似不可微的 $\|x\|_{1,2}$ 相较于基础的次梯度法可以提升算法的收敛速度.

图 1 展示了次梯度法和光滑化的梯度法的收敛曲线. 可以看出连续化策略有效加快了收敛速度。



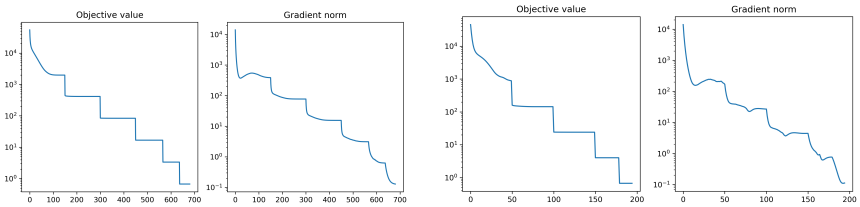
(a) 次梯度法　　(b) 光滑化的梯度法

图 1: 次梯度法和光滑化的梯度法的收敛曲线

3.2.2 近似点梯度法和 Nesterov 加速

我们发现传统的近似点梯度得到的解与次梯度或光滑化的梯度法收敛到的解的各项性质均高度相似。基于近似点梯度的优化算法使用了邻近算子处理不可微的部分，更够捕捉函数最小值附近的精确信息，因此其求解速度不仅快于基于梯度的方法，也快于商业求解器. 当使用 Nesterov 加速后，算法的收敛速度有了进一步的提升，并且收敛到了不同的解。

图 2 展示了近似点梯度法和 Nesterov 加速的收敛曲线。



(a) 近似点梯度法　　(b) Nesterov 加速的近似点梯度法

图 2: 近似点梯度法和 Nesterov 加速的近似点梯度法的收敛曲线

3.2.3 增广拉格朗日函数法

虽然对于目标函数的最优值，对对偶问题使用增广拉格朗日函数法的结果要略差于其他算法，但此时迭代次数和收敛速度优于对原问题的各种方法。

图 3 展示了增广拉格朗日函数法的收敛曲线. 从图中可以看出，目标函数的值和对偶间隙的下降曲线均较为平滑。

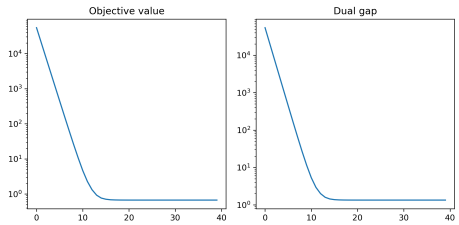


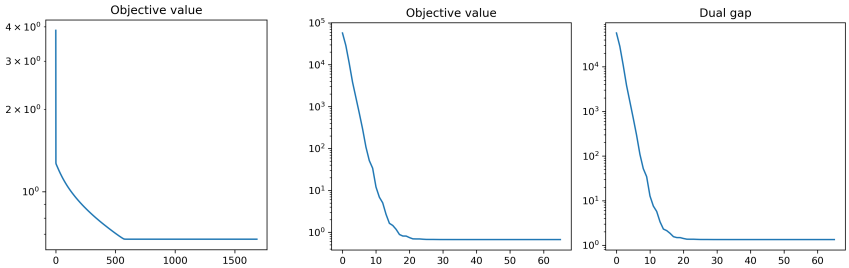
图 3: 增广拉格朗日函数法的收敛曲线

3.2.4 交替方向乘子法

对于原问题, 交替方向乘子法得到的解十分接近 Mosek 和 Gurobi 的解, 并且相比于其他算法具有更好的稀疏性。说明此方法与求解器对应的算法十分接近。

然而，使用交替方向乘子法求解对偶问题的求解速度要快于原问题，也是所有算法中最快的。但是, 与增广拉格朗日函数法相同, 由于是对转化后的对偶问题求解，交替方向乘子法的最优值要略差于其他算法，在解的稀疏性上也差于对原问题使用 ADMM 算法。

图 4 展示了交替方向乘子法的收敛曲线. 与增广拉格朗日函数法类似, 使用 ADMM 算法得到的目标函数曲线下降也较为平滑。



(a) 原问题的收敛曲线　　(b) 对偶问题的收敛曲线

图 4: 交替方向乘子法（ADMM) 的收敛曲线

4 总结

在本次作业中, 我们使用了多种算法求解 Group LASSO 问题, 并对这些算法的性能进行了评价. 从结果中可以看出, 针对问题的结构和性质设计和优化的算法往往比通用的优化算法更加高效. 在实际应用中, 我们可以根据问题的性质选择合适的算法, 从而提高算法的效率.